Rube Goldberg Machine

Description:

A Rube Goldberg Machine is a complex device that performs simple tasks in indirect and convoluted ways. We were to create a virtual Rube Goldberg Machine with ADTs like Dynamic Arrays, Queue, Stack, and Binary tree. Our program can support any number of entries. The data is read from a file and initially stored in a queue. It is then passed on to stacks and linked lists for further operations to be executed The data structures we selected are:

1. Arrays
2. Queue
3. Stack
4. Binary tree

Working of the program:

Each ADT is concerned we have the following functions:

LinkedList: LinkedList(): A constructor that initializes the head to NULL, changeHead(): Used to change the head to a given pointer, setData(): It will save new data(name, age, dob) in a new node, firstNode(): Used for creating the first node, Prepend(): Adds a new node in the front of the list, Append(): Adds a new node in the end of the list, Pop(): Removes an element from the list, popFirst(): Used to pop the head from the list, sizeVal(): Returns the size of the list, HeadVal(): Returns a pointer to the head of the list.

Stack: Push(): Pushes a new element in the stack. There are two push functions, one will take the data as argument and add it to a new node, another will take a node as an argument and at it to the stack. Pop(): Pops the topmost element in the stack, sizeVal(): Returns the size of the stack, HeadVal(): Returns a pointer to the head of the stack.

Queue: Enqueue(): Used to add a new element in the queue. There are two enqueue functions, one will take the data as argument and add it to a new node, another will take a node as an argument and at it to the queue, Dequeu(): Used to remove a queue element, sizeVal(): Returns the size of the queue, HeadVal(): Returns a pointer to the head of the queue, Disp(): Prints the queue.

BinaryTree: newNode(): Creates a new tree node, traversePreOrder(): Recursive function for preorder traversal, traverseInOrder(): Recursive function for inorder traversal, traversePostOrder(): Recursive function for postorder traversal, InsertNode(): Inserts a new node.

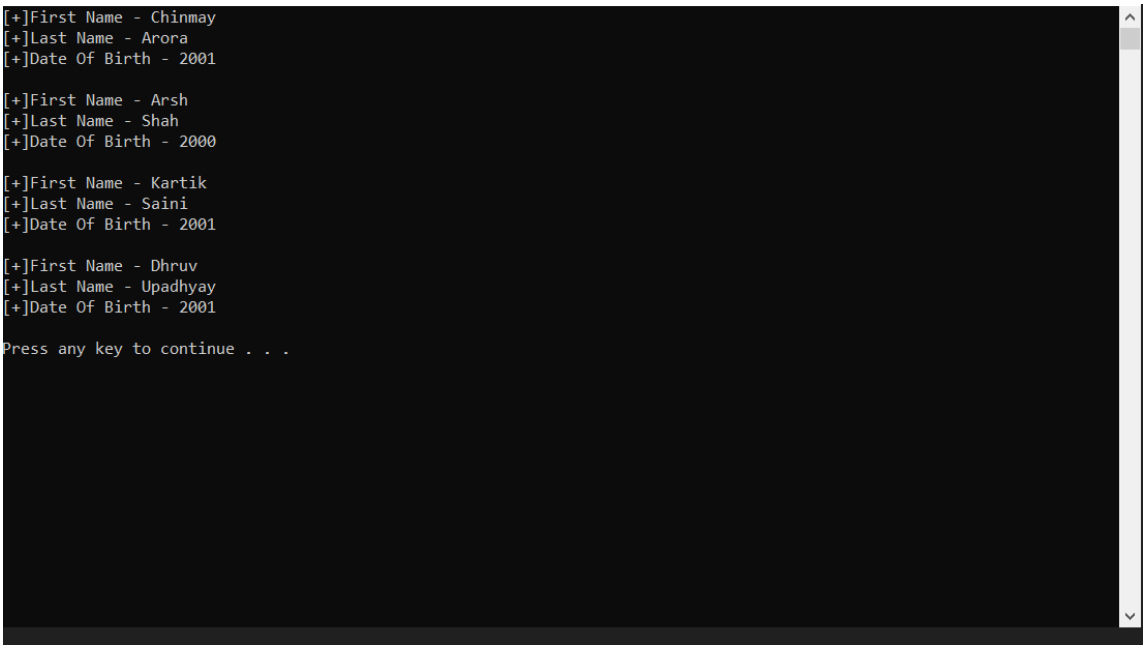
Other functions include: Partition(): Partitions the list taking the last element as the pivot, quickSortRecur(): Quick sort function using recursion, quickSort(): The main function for quicksort which calls the quickSortRecur function. This is a wrapper over the recursive function.

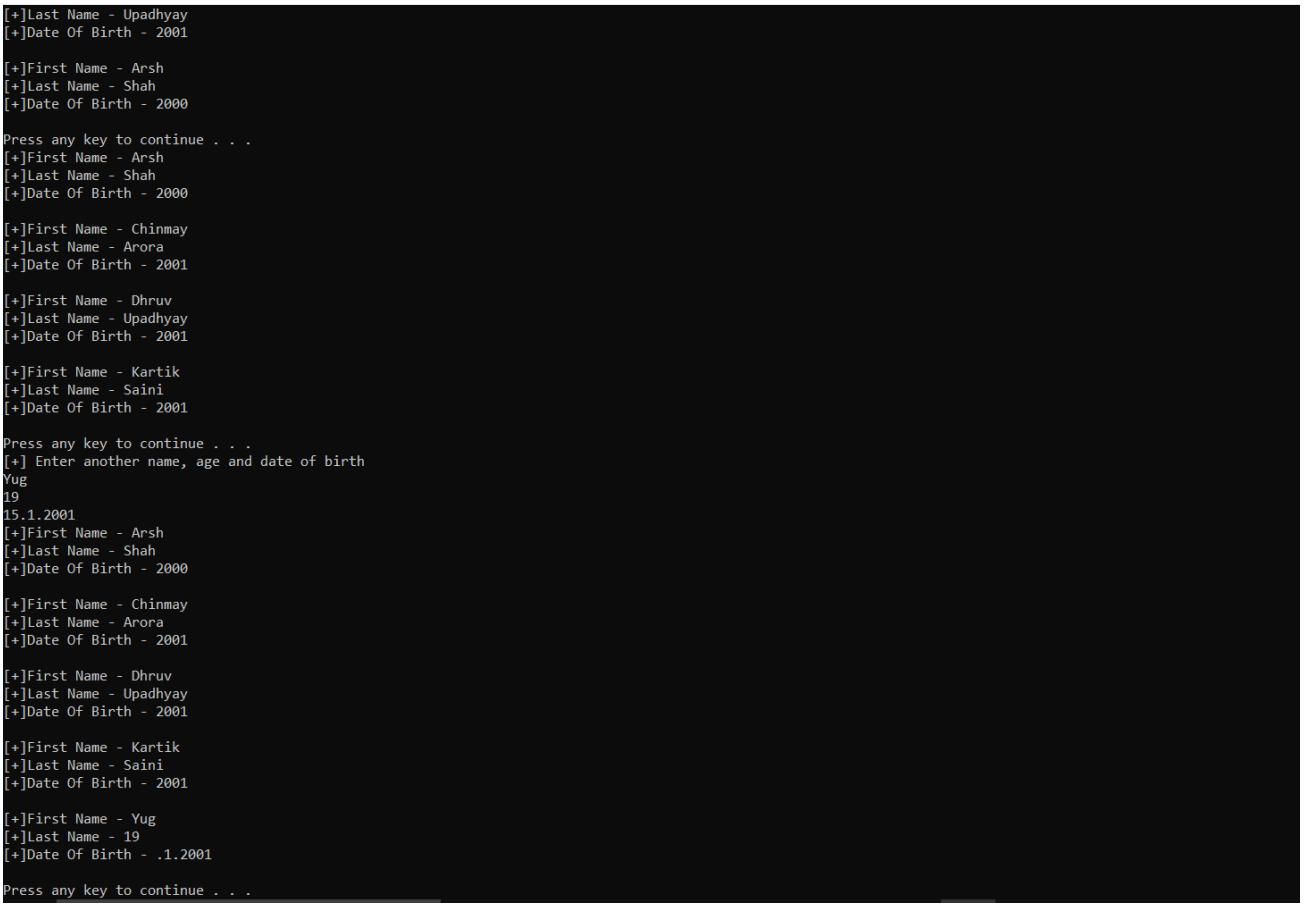
How to use our program?

1. Firstly, the input given by the user is read from a file and stored in a queue. The user is asked to press any key to read the data. Once this is initiated, the data is then displayed and hence the input is successfully stored.
2. To dequeue each element from the queue the user is asked to press any key this will proceed the program. Accordingly, the dequeued data is displayed. To continue the processing the user is then asked to press any key.
3. Now reversing the order of the data in the queue is done by dequeuing each element and pushing them onto a stack. The user is asked to press any key to dequeue and push the elements to stack. The user should press any key for the reverse order of stack to be printed.
4. After the reversed data is printed, the user is asked to press any key to pop off and requeue data one after the other. The popped elements are then printed.
5. To continue the operations, when the user again presses any key, the data from the queue is placed into an unordered binary tree. The contents of the tree are then printed in Pre-Order.
6. The user is then asked to press any key, which results in printing the contents of the tree in the Post-Order .
7. The user will be then asked to press any key to pull the contents from the tree and push it to a Linked–List using In-Order traversal. The contents of linked lists are then printed.
8. The user is asked to press any key to sort the contents of the list using a quick sort and printing it according to the birth year. 9. The contents of the list are printed and the user is asked to press any key to continue the processing. At this point, the user is done with the processing.

10. The user is asked to press 0 to final exit the program after all processing are done successfully.

Final Outputs :





Code :

```
#include <bits/stdc++.h>

#include <queue>

#include <fstream>

#include <stack>

#include <string.h>

using namespace std;



class Node // Node to represent the data in the ADTs
```

```
{
public:
    string firstName; // first Name of person
    string lastName;  // last Name of person
    int age;          // age
    string dob;       // date of birth
    Node *next;       // pointer to next Node
    Node *left;
    Node *right;
};

void display(Node *temp)
{
    cout << "[+]First Name - " << temp->firstName << endl;
    cout << "[+]Last Name - " << temp->lastName << endl;
    cout << "[+]Date Of Birth - " << temp->dob << endl
         << endl;
}

class LinkedList
{
```

```
    // ADT - Interface
    // Main Operations
    // prepend(value)
    // append(value)
    // pop()
    // popFirst()

    //head()
    //tail()

    // remove(Node)
    // remove(nodeData)
    // remove(nodePosition)
private:
    Node *head;
    int size = 0;

public:
    LinkedList()
    {
        head = NULL;
```

```
}

void changeHead(Node *val)
{
    head = val;
}

void setData(string fst, string lst, int a, string db, Node *newNode)
{
    // This saves the data into the node
    newNode->firstName = fst;
    newNode->lastName = lst;
    newNode->age = a;
    newNode->dob = db;
}
void firstNode(string fst, string lst, int a, string db)
{
    // This creates the first node
    Node *newNode = new Node();
    setData(fst, lst, a, db, newNode);
    newNode->next = head;
```

```
    head = newNode;
}

void prepend(string fst, string lst, int a, string db)
{
    size++;
    if (head == NULL)
    {
        firstNode(fst, lst, a, db);
        return;
    }
    else
    {
        Node *newNode = new Node();
        setData(fst, lst, a, db, newNode);
        newNode->next = head;
        head = newNode;
    }
}

void append(string fst, string lst, int a, string db)
```

```
{
    size++;
    if (head == NULL)
    {
        firstNode(fst, lst, a, db);
        return;
    }
    else
    {
        Node *newNode = new Node();
        setData(fst, lst, a, db, newNode);
        Node *ptr = head;
        while (ptr->next != NULL)
        {
            ptr = ptr->next;
        }
        newNode->next = ptr->next;
        ptr->next = newNode;
    }
}
```

```
Node *pop()
{
    size--;
    if (head == NULL)
    {
        cout << "[-]Empty List!!" << endl;
        return NULL;
    }
    else
    {
        Node *ptr = head, *prev = head;
        while (ptr->next != NULL)
        {
            prev = ptr;
            ptr = ptr->next;
        }
        Node *temp = ptr;
        prev->next = ptr->next;
        //delete ptr;
        return temp;
    }
```

```
}
Node *popFirst()
{
    size--;
    if (head == NULL)
    {
        return NULL;
    }
    else
    {
        Node *temp = head;
        head = head->next;
        return temp;
    }
}

int sizeVal()
{
    return size;
}
```

```
    Node *headVal()
    {
        return head;
    }
};

class Stack
{
private:
    LinkedList ll;
    int size = 0;

public:
    Stack()
    {
    }
    void push(string fst, string lst, int a, string db)
    {
        ll.prepend(fst, lst, a, db);
        size++;
    }
```

```
void push(Node *node)
{
    ll.prepend(node->firstName, node->lastName, node->age, node->dob);
    delete node;
    size++;
}
Node *pop()
{
    Node *temp = ll.popFirst();
    --size;
    return temp;
}

int sizeVal()
{
    return size;
}

Node *headVal()
{
```

```
        return ll.headVal();
    }
};

class Queue
{
private:
    LinkedList ll;
    int size = 0;

public:
    Queue() {}

    void enqueue(string fst, string lst, int a, string dob)
    {
        ll.prepend(fst, lst, a, dob);
        size++;
    }
    void enqueue(Node *node)
    {
        ll.prepend(node->firstName, node->lastName, node->age, node->dob);
```

```
    delete node;
    size++;
}
Node *dequeue()
{
    Node *temp = ll.pop();
    return temp;
    size--;
}

int sizeVal()
{
    return size;
}

Node *headVal()
{
    return ll.headVal();
}

void disp()
```

```
    {
        Node *h = ll.headVal();
        cout << h->dob;
    }
};

class BinaryTree
{

public:
    // New Node creation
    Node *newNode(string fst, string lst, int a, string db)
    {
        Node *node = new Node();
        node->firstName = fst;
        node->lastName = lst;
        node->age = a;
        node->dob = db;

        node->left = NULL;
        node->right = NULL;
```

```
    return (node);
}

void traversePreOrder(Node *temp)
{
    if (temp != NULL)
    {
        display(temp);
        traversePreOrder(temp->left);
        traversePreOrder(temp->right);
    }
}

void traverseInOrder(Node *temp)
{
    if (temp != NULL)
    {
        traverseInOrder(temp->left);
        display(temp);
        traverseInOrder(temp->right);
```

```
    }
}

void traversePostOrder(Node *temp)
{
    if (temp != NULL)
    {
        traversePostOrder(temp->left);
        traversePostOrder(temp->right);
        display(temp);
    }
}

Node *InsertNode(Node *root, string fst, string lst, int a, string db)
{
    if (root == NULL)
    {
        root = newNode(fst, lst, a, db);
        return root;
    }
    else
```

```
{
    queue<Node *> q;
    q.push(root);

    while (!q.empty())
    {
        Node *tmp = q.front();
        q.pop();

        if (tmp->left != NULL)
            q.push(tmp->left);
        else
        {
            tmp->left = newNode(fst, lst, a, db);
            return root;
        }

        if (tmp->right != NULL)
            q.push(tmp->right);
        else
        {
```

```
                tmp->right = newNode(fst, lst, a, db);
                return root;
            }
        }
    }
};
struct Node *getTail(struct Node *cur)
{
    while (cur != NULL && cur->next != NULL)
        cur = cur->next;
    return cur;
}

// Partitions the list taking the last element as the pivot
struct Node *partition(struct Node *head, struct Node *end,
                       struct Node **newHead, struct Node **newEnd)
{
    struct Node *pivot = end;
    struct Node *prev = NULL, *cur = head, *tail = pivot;
```

```
// During partition, both the head and end of the list might change
// which is updated in the newHead and newEnd variables
while (cur != pivot)
{
    if (cur->firstName < pivot->firstName)
    {
        // First node that has a value less than the pivot - becomes
        // the new head
        if ((*newHead) == NULL)
            (*newHead) = cur;

        prev = cur;
        cur = cur->next;
    }
    else // If cur node is greater than pivot
    {
        // Move cur node to next of tail, and change tail
        if (prev)
            prev->next = cur->next;
        struct Node *tmp = cur->next;
        cur->next = NULL;
```

```
            tail->next = cur;
            tail = cur;
            cur = tmp;
        }
    }

    // If the pivot data is the smallest element in the current list,
    // pivot becomes the head
    if ((*newHead) == NULL)
        (*newHead) = pivot;

    // Update newEnd to the current last node
    (*newEnd) = tail;

    // Return the pivot node
    return pivot;
}

//here the sorting happens exclusive of the end node
struct Node *quickSortRecur(struct Node *head, struct Node *end)
{
```

```
// base condition
if (!head || head == end)
    return head;

Node *newHead = NULL, *newEnd = NULL;

// Partition the list, newHead and newEnd will be updated
// by the partition function
struct Node *pivot = partition(head, end, &newHead, &newEnd);

// If pivot is the smallest element - no need to recur for
// the left part.
if (newHead != pivot)
{
    // Set the node before the pivot node as NULL
    struct Node *tmp = newHead;
    while (tmp->next != pivot)
        tmp = tmp->next;
    tmp->next = NULL;

    // Recur for the list before pivot
```

```
        newHead = quickSortRecur(newHead, tmp);

        // Change next of last node of the left half to pivot
        tmp = getTail(newHead);
        tmp->next = pivot;
    }

    // Recur for the list after the pivot element
    pivot->next = quickSortRecur(pivot->next, newEnd);

    return newHead;
}

// The main function for quick sort. This is a wrapper over recursive
// function quickSortRecur()
void quickSort(struct Node **headRef)
{
    (*headRef) = quickSortRecur(*headRef, getTail(*headRef));
    return;
}
```

```
int main()

{

    // Setting up a queue

    Queue q;



    // Reading input from the text file

    // Reads one line at a time and divides the line

    // with respect to comma and space

//~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~//

    ifstream fin;

    fin.open("test.txt");

    string line;

    while (fin)

    {

        getline(fin, line);

        string delimiter = ", ";

        size_t pos = 0;

        string token;

        string arr[4];

        int i = 0;
```

```
while ((pos = line.find(delimiter)) != string::npos)
{
    token = line.substr(0, pos);
    arr[i] = token;
    line.erase(0, pos + delimiter.length());
    ++i;
}
arr[3] = line;
arr[2] = arr[1];
istringstream ss(arr[0]);
string word;
i = 0;
while (ss >> word)
{
    arr[i] = word;
    i++;
}
stringstream num(arr[2]);
int x = 0;
num >> x;
```

```
        q.enqueue(arr[0], arr[1], x, arr[3]);

//~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~//

    }

    // Step 1
    Queue qB;
    int s = q.sizeVal() - 1;
    while (s != 0)
    {
        Node *temp = q.dequeue();
        display(temp);
        qB.enqueue(temp);
        --s;
    }

    // Wait for user input
    system("pause");

    // Step 2
```

```
Stack st;
s = qB.sizeVal();
while (s != 0)
{
    Node *tmp = qB.dequeue();
    st.push(tmp);
    --s;
}

Queue qC;
s = st.sizeVal();
while (s != 0)
{
    Node *tmp = st.pop();
    qC.enqueue(tmp);
    s--;
}

Queue qD;
s = qC.sizeVal();
while (s != 0)
```

```
{
    Node *tmp = qC.dequeue();
    display(tmp);
    qD.enqueue(tmp);
    --s;
}

system("pause");
// Step - 3
BinaryTree bt;
Node *root = NULL;
s = qD.sizeVal();

while (s != 0)
{
    Node *tmp = qD.dequeue();
    root = bt.InsertNode(root, tmp->firstName, tmp->lastName, tmp->age, tmp->dob);
    --s;
}
cout << root->age;
bt.traversePreOrder(root);
```

```
system("pause");


bt.traversePostOrder(root);


system("pause");


// Transferring the data from the binary tree
// linked list using in order traversal
LinkedList lt;
stack<Node *> stac;
Node *curr = root;


while (curr != NULL || stac.empty() == false)
{
   while (curr != NULL)
   {
      stac.push(curr);
      curr = curr->left;
   }
   curr = stac.top();
```

```
    stac.pop();
    lt.append(curr->firstName, curr->lastName, curr->age, curr->dob);
    curr = curr->right;
}

Node *hd = lt.headVal();
while (hd != NULL)
{
    display(hd);
    hd = hd->next;
}

system("pause");

// Step - 4
Node *a = lt.headVal();
quickSort(&a);

Node *hdt = a;
while (hdt != NULL)
{
```

```
    display(hdt);
    hdt = hdt->next;
}

system("pause");
string f, l, d;
int ag;
cout << "[+] Enter another name, age and date of birth\n";
cin >> f >> l >> ag >> d;
lt.changeHead(a);
lt.append(f, l, ag, d);
a = lt.headVal();
quickSort(&a);
lt.changeHead(a);
hdt = a;
while (hdt != NULL)
{
    display(hdt);
    hdt = hdt->next;
}
system("pause");
```

```
cout << "Bye Bye!!";

return 0;
```

Group Members:

1. Arsh Shah (RA1911003010286)
2. Kartik Saini (RA1911003010295)
3. Chinmay Arora (RA1911003010298)
4. Dhruv Upadhyay (RA1911003010303)